MATP: Temporal and State Aware Semantic Memory for Agentic LLMs

Contribution 1: Versioning with a Persistent Memory Buffer via MCP

Rahul Saravanan

Department of Information Technology
National Institute of Technology Karnataka, Surathkal

End-Semester Evaluation, November 2026

The long horizon problem

What breaks today

- Agent keeps 10 to 30 tool outputs in context. FIFO eviction drops early, critical state and causes catastrophic forgetting.
- Naive RAG keeps everything but has no notion of mutation. A query still matches schema v1 on keyword overlap and causes stale state hallucination.

What we need

- Keep all history, but retrieve only the current version of each resource.
- Suppress states that were *derived from* mutated resources, not just the mutated file itself.
- No token bloat, no external service, no network.

Question: how to retain history without returning superseded or dependent-stale state?

Objectives

Goal

Replace rigid context pruning with a persistent, queryable semantic memory buffer exposed as an MCP server.

Dual memory

- Working memory: small active context for reasoning.
- Long term buffer: keeps every version, tagged by resource id and logical clock.

Operations

- Ingest: tag, embed, version, mark superseded, record dependency edges.
- Retrieve: score with $R(q, v_i)$, filter stale, decay by age.
- Audit: full history and dependency graph per resource.

Scoring: $R(q, v_i)$

State aware retrieval

Let q be the query embedding, v_i a stored state, t_{curr} the logical clock, t_i the birth step of v_i:

$$R(q, v_i) = \begin{cases} 0 & \text{if } \exists j:\ \text{id}_j = \text{id}_i \wedge t_j > t_i \\ 0 & \text{if } v_i \text{ depends on a state mutated after } t_i \\ \cos(q, v_i) \cdot e^{-\lambda\,(t_{\text{curr}} - t_i)} & \text{otherwise} \end{cases}$$

1. Logical clock t

Discrete causal steps, not wall time. Each ingest increments the clock.

2. Superseding $R = 0$

Newer version of the same resource zeroes the old one when filtering is enabled.

3. Decay $e^{-\lambda \Delta t}$

Among active states, recent steps outrank old ones. Default $\lambda = 0.05$.

Dependency graph invalidation (endsem)

Cross-resource stale state

- Midsem leak: ORM `user.py` v1 kept scoring after schema v2 because its resource id differs.
- Fix: ingest declares `dependencies=[resource_id, ...]`.
- If any dependency has a newer version than v_i, v_i is also $R = 0$.
- Propagation is transitive: `tests` $\rightarrow$ `session` $\rightarrow$ `auth`.

Lifecycle

- Re-ingesting the dependent after the dependency refresh clears the invalidation (its step $>$ dependency step).
- Edges persist in the JSONL metadata; `dependency_graph()` MCP tool exposes nodes, edges, dependents, invalid set.
- On the 40-step benchmark the graph suppresses `tests/test_auth.py` (written against session v2) when session v3 lands.

Adaptive temporal decay (endsem)

Learn λ per task instead of one global constant

Feedback reports which retrieved items were useful at age Δt_i:

$$\lambda_{\text{task}} \leftarrow (1-\alpha)\,\lambda_{\text{task}} + \alpha\,\frac{\ln 2}{\operatorname{median}(\Delta t_i)}$$

The half-life is set so the median useful state decays by 50%.

Interface

`time_decay="auto"`, `task="..."` on retrieve; `record_feedback(task, ids)` after use.

Behaviour

Always-fresh tasks grow λ (strong recency); horizon tasks shrink it. Clipped to $[0.005, 0.5]$.

Persistence

Stored in `<db>.decay.json`; `get_decay_stats()` reports learned values.

Benchmark: long horizon ($t = 1$ to 40)

Setup: 10 resources, 3 schema versions, 2 auth versions, ORM/session/routes/tests declared as dependents; 5 queries with legacy-heavy terms so lexical overlap favours stale text. FIFO baseline keeps a 10-step window.

Method	Lineage stale (top-5)	Graph-drop	Precision	Retention
FIFO (window 10)	–	–	0% (2/4 forgotten)	50%
Naive RAG (cosine, $k = 5$)	$20/25 = 80\%$	–	$5/25 = 20\%$	100%
MATP (lineage, $k = 5$, $\lambda = 0.05$)	$0/25 = 0\%$	–	$25/25 = 100\%$	100%
MATP+graph (this work)	**0/25 = 0%**	**1**	**25/25 = 100%**	**100%**

What the numbers mean

- Naive surfaces 4 of 5 stale versions per query: the midsem leak generalises.
- Lineage filtering alone already reaches 100% here; the graph additionally kills the dependent `tests/test_auth.py`.
- FIFO would forget `db/schema.sql` and `config/auth.env` entirely.

Scenario demo (midsem setup, 16 steps)

- Naive: 2/5 stale (schema v1 AND auth v1 in top-5).
- MATP lineage-only: 0 stale, but ORM v1 leaks (3/4 precision).
- MATP+graph: ORM suppressed, 3/3 precision; decay sweep and MCP round trip still pass.

Status and next steps

Done

- Grounded $R(q, v_i)$ with clock, superseding and decay.
- Persistent buffer with lineage audit (atomic writes, clock persistence).
- MCP server over stdio with local embeddings; 11 tools, 4 resources.
- Cross-resource dependency graph with transitive invalidation.
- Adaptive per-task decay from retrieval feedback.
- Long-horizon benchmark: 80% $\rightarrow$ 0% stale in top-5.

Next (beyond scope)

- Learn dependency edges automatically (read diffs / call graph) instead of agent-declared.
- Multi-task λ routing by subgoal detection in the planner.
- Approximate indexes (IVF) and streaming ingestion for 100k+ states.
- Full agent-in-the-loop evaluation on longer SWE-style workloads.

Takeaway

- Retain without bloat: long term buffer keeps every version, working memory stays small.
- Retrieve without stale: $R = 0$ covers superseded versions *and* states whose dependency chain moved on.
- Recall at the right horizon: λ is learned per task from feedback, not hand-tuned.
- Deploy without infra: stdio MCP, local embeddings, single file persistence.

Thank you

Questions and discussion